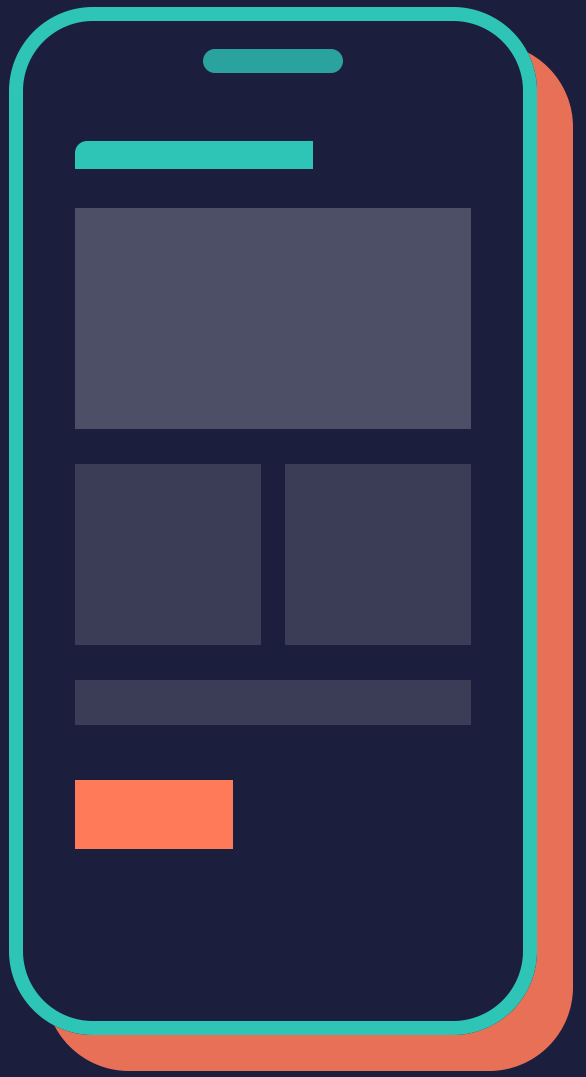
Architecture MVC et état

Cours 2 – Modèles, contrôleurs, vues et Context

Chris CHEVALIER



Sommaire

01 Pourquoi une architecture ?

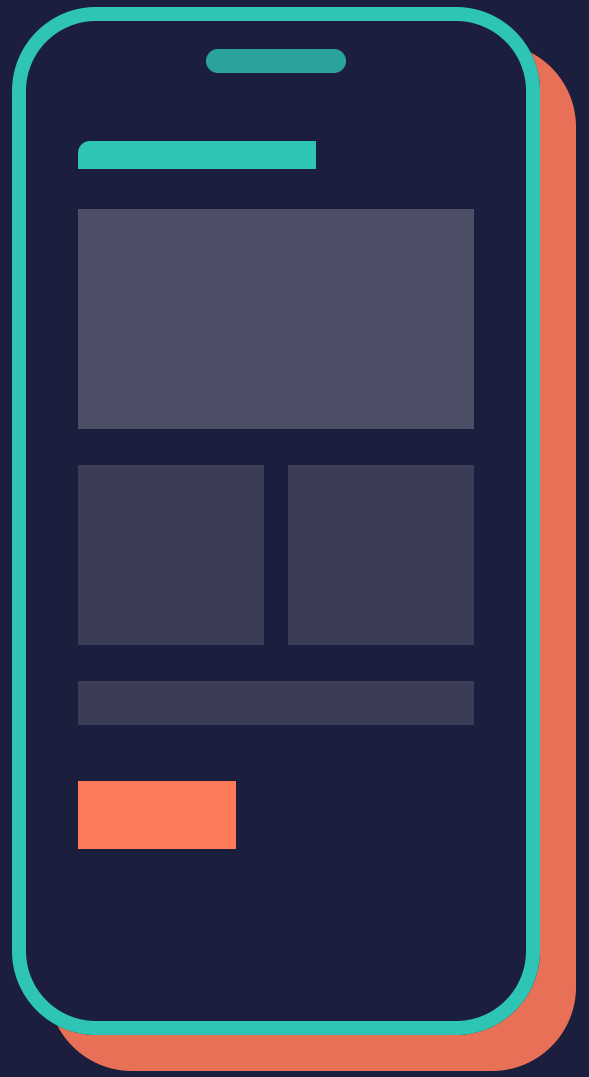
02 Le pattern MVC dans React

03 Le modèle : types et règles

04 Le contrôleur : services et hooks

05 La vue : des composants purs

06 L'état local et l'état global



01

Pourquoi une architecture ?

Le problème de l'écran « tout-en-un »



Un écran qui fait tout

src/app/index.tsx

```tsx
// AVANT le refactoring : tout est dans l'écran (exemple à ne pas
// suivre)
import { useEffect, useState } from "react";
import {
  ActivityIndicator,
  FlatList,
  Pressable,
  Text,
  TextInput,
  View,
} from "react-native";

import { STAYS } from "@/data/stays";
import { Stay } from "@/models/stay";

export default function CatalogScreenBefore() {
  const [stays, setStays] = useState<Stay[]>([]);
  const [loading, setLoading] = useState(true);
  const [query, setQuery] = useState("");
  const [favorites, setFavorites] = useState<string[]>([]);
```

- Le chargement, les règles de recherche, les favoris et l'affichage sont **mélangés**
- Rien n'est **réutilisable** dans un autre écran
- Rien n'est **testable** sans afficher l'écran
- Remplacer les données locales par une API oblige à **réécrire l'écran**

Un écran qui fait tout (suite)

src/app/index.tsx

```tsx

  useEffect(() => {
    setTimeout(() => {
      setStays(STAYS);
      setLoading(false);
    }, 600);
  }, []);

  if (loading) return <ActivityIndicator />;

  return (
    <View style={{ flex: 1, padding: 8 }}>
      <TextInput
        value={query}
        onChangeText={setQuery}
        placeholder="Rechercher"
      />
      <FlatList
        data={stays.filter((s) => s.title.includes(query))}
        keyExtractor={(s) => s.id}
```

Un écran qui fait tout (suite)

src/app/index.tsx

```
41          renderItem={({ item }) => (
42            <Pressable
43              onPress={() =>
44                setFavorites(
45                  favorites.includes(item.id)
46                    ? favorites.filter((f) => f !== item.id)
47                    : [...favorites, item.id],
48                )
49              }
50            >
51              <Text>
52                {favorites.includes(item.id) ? "★" : "☆"} {item.title} ·{" "}
53                {item.pricePerNight} €
54              </Text>
55            </Pressable>
56          )}
57        />
58      </View>
59    );
60  }
```

Une responsabilité par fichier

> **Principe de responsabilité unique** (le « S » de SOLID) : un module ne doit avoir **qu'une seule raison de changer**.

Les données changent

On passe des données locales à une API. Seul le **service** change

Une règle change

La recherche doit aussi porter sur la ville. Seul le **modèle** change

Le design change

Les cartes deviennent des lignes. Seule la **vue** change

02

Le pattern MVC dans React

Modèle, vue, contrôleur



MVC : trois rôles

Rôle	Ce qu'il fait	Dans notre projet
Modèle	Décrit les données et les règles métier, sans rien afficher	`src/models/` : types et fonctions pures `src/services/` : accès aux données
Contrôleur	Tient l'état, réagit aux actions de l'utilisateur, appelle les services	`src/hooks/` : hooks personnalisés `src/context/` : état partagé
Vue	Affiche ce qu'on lui donne et signale les actions	`src/components/` : composants purs `src/app/` : écrans qui assemblent

> MVC vient des interfaces graphiques des années 1970. Dans React, le contrôleur n'est pas une classe : c'est un **hook personnalisé**.

> **Erreur fréquente** : écrire la logique métier dans le hook. Les règles vivent dans le **modèle**, l'accès aux données dans les **services**. Le contrôleur ne fait qu'orchestrer : il les appelle et tient l'état.

Le trajet d'une action

1. Vue

L'utilisateur tape « Lyon ». `StayFiltersBar` appelle `onChange({ query: "Lyon" })`

2. Contrôleur

`useStayFilters` met à jour son état `filters`

3. Modèle

`matchesFilters()` décide quels séjours correspondent

4. Vue

React affiche à nouveau la liste avec les séjours visibles

> Les données **descendent** par les props, les événements **remontent** par les fonctions de rappel. La vue ne modifie jamais les données elle-même.

L'arborescence cible

src/

```
src/
├── app/                    VUE : écrans
│   ├── _layout.tsx         fournit le Context
│   └── index.tsx           catalogue
├── components/             VUE : composants purs
│   ├── stay-card.tsx
│   ├── stay-filters-bar.tsx
│   ├── state-message.tsx
│   └── favorites-badge.tsx
├── hooks/                  CONTRÔLEUR
│   ├── use-stays.ts
│   ├── use-stay-filters.ts
│   └── use-favorites.ts
├── context/                CONTRÔLEUR partagé
│   └── favorites-context.tsx
├── services/               MODÈLE : accès aux données
│   └── stays-service.ts
├── models/                 MODÈLE : types et règles
│   └── stay.ts
└── data/stays.ts           données locales (cours 1)
```

- Un fichier par **domaine** : les séjours, les favoris…
- Noms de fichiers en **kebab-case** (convention d'Expo), composants en PascalCase, hooks en `useXxx`
- C'est la structure attendue pour le projet

03

Le modèle : types et règles

Des données décrites et des fonctions pures



Modéliser les données

src/models/stay.ts

```ts
export type StayType = "apartment" | "house" | "cabin";

export type Stay = {
  id: string;
  title: string;
  city: string;
  type: StayType;
  pricePerNight: number;
  available: boolean;
  latitude: number;
  longitude: number;
  imageUrl?: string;
};

export type StayFilters = {
  query: string;
  type: StayType | null;
  onlyAvailable: boolean;
};
```

- Les types sont le **contrat** entre toutes les couches
- `StayType | null` : « aucun type choisi » est une valeur prévue
- `Record<StayType, string>` : un objet avec **une entrée par type**. En ajouter un sans libellé est une erreur

Modéliser les données (suite)

src/models/stay.ts

```ts
export const STAY_TYPE_LABELS: Record<StayType, string> = {
  apartment: "Appartement",
  house: "Maison",
  cabin: "Cabane",
};

// Règle métier : une fonction pure, sans React, facile à tester
export function matchesFilters(
  stay: Stay,
  filters: StayFilters,
 ): boolean {
  const search = filters.query.trim().toLowerCase();
  const matchesQuery =
    stay.title.toLowerCase().includes(search) ||
    stay.city.toLowerCase().includes(search);
  const matchesType =
    filters.type === null || stay.type === filters.type;
  const matchesAvailability =
    !filters.onlyAvailable || stay.available;
  return matchesQuery && matchesType && matchesAvailability;
```

Modéliser les données (suite)

src/models/stay.ts

```
41 }
```

Les règles métier : des fonctions pures

src/models/stay.ts

```ts
export type StayType = "apartment" | "house" | "cabin";

export type Stay = {
  id: string;
  title: string;
  city: string;
  type: StayType;
  pricePerNight: number;
  available: boolean;
  latitude: number;
  longitude: number;
  imageUrl?: string;
};

export type StayFilters = {
  query: string;
  type: StayType | null;
  onlyAvailable: boolean;
};
```

- **Pure** : le même résultat pour les mêmes paramètres, sans effet de bord
- Elle ne connaît ni React ni l'écran : on peut la **tester** en une ligne et la **réutiliser** partout
- Une règle change ? On modifie cette fonction, et seulement elle

Les règles métier : des fonctions pures (suite)

src/models/stay.ts

```ts
export const STAY_TYPE_LABELS: Record<StayType, string> = {
  apartment: "Appartement",
  house: "Maison",
  cabin: "Cabane",
};

// Règle métier : une fonction pure, sans React, facile à tester
export function matchesFilters(
  stay: Stay,
  filters: StayFilters,
 ): boolean {
  const search = filters.query.trim().toLowerCase();
  const matchesQuery =
    stay.title.toLowerCase().includes(search) ||
    stay.city.toLowerCase().includes(search);
  const matchesType =
    filters.type === null || stay.type === filters.type;
  const matchesAvailability =
    !filters.onlyAvailable || stay.available;
  return matchesQuery && matchesType && matchesAvailability;
```

Les règles métier : des fonctions pures (suite)

src/models/stay.ts

```
41 }
```

04

Le contrôleur : services et hooks

Charger, filtrer, réagir



Le service : appeler l'API et convertir

src/services/stays-service.ts

```ts
import { Stay, StayType } from "@/models/stay";

const SITE = "https://chris-chevalier.github.io/react-native-supports";
const STAYS_URL = `${SITE}/api/stays.json`;

// Forme de la réponse de l'API, limitée à ce qu'on utilise
type StayResponse = {
  id: string;
  title: string;
  type: StayType;
  price_per_night: number;
  is_available: boolean;
  image_url: string | null;
  location: { city: string; latitude: number; longitude: number };
};

// Traduit une réponse de l'API dans le vocabulaire du modèle
function toStay(response: StayResponse): Stay {
  return {
    id: response.id,
```

- `StayResponse` décrit la réponse de l'API avec **ses** noms : `price_per_night`, `location.city`…
- `toStay` la **traduit** dans le vocabulaire du modèle. Le reste de l'application ne connaît jamais les noms de l'API
- `fetch` fait un vrai appel réseau. Il ne lève pas d'erreur sur une réponse 404 ou 500 : testez `response.ok`
- Le service tient sa promesse : `Promise<Stay[]>`. Si l'API change, **seul ce fichier change**

Le service : appeler l'API et convertir (suite)

src/services/stays-service.ts

```
21      title: response.title,
22      city: response.location.city,
23      type: response.type,
24      pricePerNight: response.price_per_night,
25      available: response.is_available,
26      latitude: response.location.latitude,
27      longitude: response.location.longitude,
28      imageUrl: response.image_url ?? undefined,
29    };
30  }
31
32  export async function fetchStays(): Promise<Stay[]> {
33    const response = await fetch(STAYS_URL);
34    if (!response.ok) throw new Error(`Erreur HTTP ${response.status}`);
35    const data: StayResponse[] = await response.json();
36    return data.map(toStay);
37  }
```

Un état qui ne peut pas mentir

src/hooks/use-stays.ts

```ts
import { useEffect, useState } from "react";

import { Stay } from "@/models/stay";
import { fetchStays } from "@/services/stays-service";

export type StaysState =
  | { status: "loading" }
  | { status: "error"; message: string }
  | { status: "success"; stays: Stay[] };

// Contrôleur : charge les séjours et expose un état que la vue sait
// afficher
export function useStays() {
  const [state, setState] = useState<StaysState>({
    status: "loading",
  });
  const [attempt, setAttempt] = useState(0);

  useEffect(() => {
    let cancelled = false;
```

- Une **union discriminée** : la propriété `status` indique la forme de l'objet
- Impossible d'avoir à la fois une erreur et des données. Avec trois booléens (`loading`, `error`, `data`), huit combinaisons existent, dont cinq absurdes
- Dans l'écran, après `if (state.status === "error") return …`, TypeScript sait que `state` ne peut plus être qu'en chargement ou en succès

Un état qui ne peut pas mentir (suite)

src/hooks/use-stays.ts

```
21      fetchStays()
22        .then((stays) => {
23          if (!cancelled) setState({ status: "success", stays });
24        })
25        .catch(() => {
26          if (!cancelled)
27            setState({
28              status: "error",
29              message: "Échec du chargement",
30            });
31        });
32      return () => {
33        cancelled = true;
34      };
35    }, [attempt]);
36
37    const reload = () => {
38      setState({ status: "loading" });
39      setAttempt((n) => n + 1);
40    };
```

Un état qui ne peut pas mentir (suite)

src/hooks/use-stays.ts

```
41
42   return { state, reload };
43 }
```

Le hook contrôleur `useStays`

src/hooks/use-stays.ts

```ts
import { useEffect, useState } from "react";

import { Stay } from "@/models/stay";
import { fetchStays } from "@/services/stays-service";

export type StaysState =
  | { status: "loading" }
  | { status: "error"; message: string }
  | { status: "success"; stays: Stay[] };

// Contrôleur : charge les séjours et expose un état que la vue sait
// afficher
export function useStays() {
  const [state, setState] = useState<StaysState>({
    status: "loading",
  });
  const [attempt, setAttempt] = useState(0);

  useEffect(() => {
    let cancelled = false;
```

- Un **hook personnalisé** est une fonction `useXxx` qui utilise d'autres hooks
- `cancelled` évite de modifier l'état d'un écran déjà quitté
- Changer `attempt` relance l'effet : c'est le bouton « Réessayer »
- Le hook renvoie **ce dont la vue a besoin**, rien de plus

Le hook contrôleur useStays (suite)

src/hooks/use-stays.ts

```
    fetchStays()
      .then((stays) => {
        if (!cancelled) setState({ status: "success", stays });
      })
      .catch(() => {
        if (!cancelled)
          setState({
            status: "error",
            message: "Échec du chargement",
          });
      });
    return () => {
      cancelled = true;
    };
  }, [attempt]);

  const reload = () => {
    setState({ status: "loading" });
    setAttempt((n) => n + 1);
  };
```

Le hook contrôleur `useStays` (suite)

src/hooks/use-stays.ts

```
41
42   return { state, reload };
43 }
```

Le hook `useStayFilters` : un état dérivé

src/hooks/use-stay-filters.ts

```ts
import { useState } from "react";

import { matchesFilters, Stay, StayFilters } from "@/models/stay";

const INITIAL_FILTERS: StayFilters = {
  query: "",
  type: null,
  onlyAvailable: false,
};

// Contrôleur : l'état des filtres et la liste qui en découle
export function useStayFilters(stays: Stay[]) {
  const [filters, setFilters] =
    useState<StayFilters>(INITIAL_FILTERS);

  const updateFilters = (changes: Partial<StayFilters>) =>
    setFilters((current) => ({ ...current, ...changes }));

  const visibleStays = stays.filter((stay) =>
    matchesFilters(stay, filters),
```

- `visibleStays` n'est **pas un état** : il se calcule à chaque rendu
- Stocker une donnée calculable crée deux sources de vérité qui finissent par diverger
- `Partial<StayFilters>` : toutes les propriétés deviennent facultatives
- Le React Compiler, activé par Expo, évite les calculs inutiles : pas besoin de `useMemo`

Le hook `useStayFilters` : un état dérivé (suite)

src/hooks/use-stay-filters.ts

```
21   );
22
23   return { filters, updateFilters, visibleStays };
24 }
```

Quiz

Je dois trier les séjours par prix. Où écrire la fonction de tri ?

> Dans le modèle (`src/models/stay.ts`) : c'est une règle métier, pure et testable. Le hook l'appelle, la vue n'en sait rien.

Faut-il un `useState` pour le nombre de séjours visibles ?

> Non : `visibleStays.length` se calcule. Un état ne doit contenir que ce qui ne se déduit de rien d'autre.

05

La vue : des composants purs

Afficher et signaler, rien d'autre



Un composant d'affichage pur

src/components/stay-filters-bar.tsx

```tsx
import {
  Pressable,
  StyleSheet,
  Switch,
  Text,
  TextInput,
  View,
} from "react-native";

import {
  STAY_TYPE_LABELS,
  StayFilters,
  StayType,
} from "@/models/stay";

type StayFiltersBarProps = {
  filters: StayFilters;
  onChange: (changes: Partial<StayFilters>) => void;
};
```

- Il n'assemble que des **composants de base** (`TextInput` , `Pressable` , `Switch`) et leurs styles
- Il **reçoit** les filtres et **signale** les changements
- Aucun état, aucun service, aucun `useEffect`
- On peut le réutiliser ailleurs ou changer son design sans toucher à la logique

Un composant d'affichage pur (suite)

src/components/stay-filters-bar.tsx

```tsx
const TYPES = Object.keys(STAY_TYPE_LABELS) as StayType[];

// Vue pure : affiche les filtres reçus et signale chaque changement
export function StayFiltersBar({
  filters,
  onChange,
}: StayFiltersBarProps) {
  return (
    <View style={styles.bar}>
      <TextInput
        value={filters.query}
        onChangeText={(query) => onChange({ query })}
        placeholder="Rechercher une ville ou un logement"
        style={styles.search}
      />
      <View style={styles.row}>
        {TYPES.map((type) => (
          <Pressable
            key={type}
            onPress={() =>
```

Un composant d'affichage pur (suite)

src/components/stay-filters-bar.tsx

```tsx
            onChange({ type: filters.type === type ? null : type })
          }
          style={[
            styles.chip,
            filters.type === type && styles.chipSelected,
          ]}
        >
          <Text style={styles.chipLabel}>
            {STAY_TYPE_LABELS[type]}
          </Text>
        </Pressable>
      ))}
      <View style={styles.switch}>
        <Switch
          value={filters.onlyAvailable}
          onValueChange={(onlyAvailable) =>
            onChange({ onlyAvailable })
          }
        />
        <Text>Disponibles</Text>
```

Un composant d'affichage pur (suite)

src/components/stay-filters-bar.tsx

```
61          </View>
62        </View>
63      </View>
64    );
65  }
66
67  const styles = StyleSheet.create({
68    bar: { gap: 8, margin: 8 },
69    search: {
70      padding: 12,
71      borderRadius: 8,
72      borderWidth: 1,
73      borderColor: "#D9DCEA",
74      backgroundColor: "#fff",
75      fontSize: 16,
76    },
77    row: {
78      flexDirection: "row",
79      flexWrap: "wrap",
80      alignItems: "center",
```

Un composant d'affichage pur (suite)

src/components/stay-filters-bar.tsx

```
    gap: 8,
  },
  chip: {
    paddingVertical: 6,
    paddingHorizontal: 14,
    borderRadius: 20,
    borderWidth: 1,
    borderColor: "#D9DCEA",
    backgroundColor: "#fff",
  },
  chipSelected: {
    backgroundColor: "#2EC4B6",
    borderColor: "#2EC4B6",
  },
  chipLabel: { fontSize: 14, fontWeight: "600" },
  switch: { flexDirection: "row", alignItems: "center", gap: 6 },
});
```

Un autre composant pur : `StateMessage`

src/components/state-message.tsx

```tsx
import {
  ActivityIndicator,
  Pressable,
  StyleSheet,
  Text,
  View,
} from "react-native";

type StateMessageProps = {
  loading?: boolean;
  message?: string;
  onRetry?: () => void;
};

export function StateMessage({
  loading = false,
  message,
  onRetry,
}: StateMessageProps) {
  return (
```

- Trois affichages possibles : un indicateur de chargement, un message, un bouton « Réessayer »
- Il ne sait pas **pourquoi** il s'affiche : c'est l'écran qui décide
- Toutes les props sont facultatives : `<StateMessage loading />` suffit pendant un chargement

Un autre composant pur : `StateMessage` (suite)

src/components/state-message.tsx

```tsx
    <View style={styles.box}>
      {loading && <ActivityIndicator size="large" />}
      {message && <Text style={styles.message}>{message}</Text>}
      {onRetry && (
        <Pressable onPress={onRetry} style={styles.button}>
          <Text style={styles.buttonLabel}>Réessayer</Text>
        </Pressable>
      )}
    </View>
  );
}

const styles = StyleSheet.create({
  box: {
    flex: 1,
    alignItems: "center",
    justifyContent: "center",
    gap: 12,
    padding: 24,
  },
```

Un autre composant pur : `StateMessage` (suite)

src/components/state-message.tsx

```
41    message: { fontSize: 16, textAlign: "center" },
42    button: {
43      paddingVertical: 10,
44      paddingHorizontal: 18,
45      borderRadius: 8,
46      backgroundColor: "#E2F7F4",
47    },
48    buttonLabel: { fontWeight: "600" },
49  });
```

L'écran assemblé

src/app/index.tsx

```tsx
import { useState } from "react";
import { FlatList, StyleSheet, Text, View } from "react-native";

import { StateMessage } from "@/components/state-message";
import { StayCard } from "@/components/stay-card";
import { StayFiltersBar } from "@/components/stay-filters-bar";
import { useColumns } from "@/hooks/use-columns";
import { useStayFilters } from "@/hooks/use-stay-filters";
import { useStays } from "@/hooks/use-stays";

// Écran : il assemble les contrôleurs (hooks) et les vues
// (composants)
export default function CatalogScreen() {
  const columns = useColumns();
  const { state, reload } = useStays();
  const stays = state.status === "success" ? state.stays : [];
  const { filters, updateFilters, visibleStays } =
    useStayFilters(stays);
  const [favorites, setFavorites] = useState<string[]>([]);
  const isFavorite = (id: string) => favorites.includes(id);
```

- L'écran ne contient **aucune règle** : il branche les hooks sur les composants
- Il gère les trois états : chargement, erreur, succès
- Pour l'instant, les favoris sont un état local de l'écran
- Les hooks sont appelés **avant** tout `return` : c'est une règle de React

L'écran assemblé (suite)

src/app/index.tsx

```tsx
  const toggleFavorite = (id: string) =>
    setFavorites((current) =>
      current.includes(id)
        ? current.filter((f) => f !== id)
        : [...current, id],
    );

  if (state.status === "loading") return <StateMessage loading />;
  if (state.status === "error")
    return <StateMessage message={state.message} onRetry={reload} />;

  return (
    <View style={styles.container}>
      <View style={styles.header}>
        <Text style={styles.heading}>Séjours</Text>
        <Text style={styles.count}>{favorites.length} favori(s)</Text>
      </View>
      <StayFiltersBar filters={filters} onChange={updateFilters} />
      <FlatList
        key={columns}
```

L'écran assemblé (suite)

src/app/index.tsx

```
41          numColumns={columns}
42          data={visibleStays}
43          keyExtractor={(stay) => stay.id}
44          renderItem={({ item }) => (
45            <StayCard
46              stay={item}
47              isFavorite={isFavorite(item.id)}
48              onToggleFavorite={toggleFavorite}
49            />
50          )}
51          ListEmptyComponent={
52            <Text style={styles.empty}>
53              Aucun logement ne correspond.
54            </Text>
55          }
56        />
57      </View>
58    );
59  }
```

L'écran assemblé (suite)

src/app/index.tsx

```tsx
const styles = StyleSheet.create({
  container: {
    flex: 1,
    width: "100%",
    maxWidth: 1100,
    alignSelf: "center",
    padding: 8,
  },
  header: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    margin: 8,
  },
  heading: { fontSize: 24, fontWeight: "800" },
  count: { fontSize: 16, fontWeight: "600", color: "#0B7A70" },
  empty: { margin: 8, fontSize: 16 },
});
```

Check-list d'une vue pure

Une vue pure…

- reçoit ses données par des **props typées**
- signale les actions par des **fonctions de rappel** (`onChange`, `onPress`…)
- peut avoir un petit état **d'interface** (un menu ouvert, un champ en cours de saisie)

Une vue pure ne…

- n'importe **aucun service** et n'appelle ni `fetch` ni Firebase
- ne contient **aucune règle métier** (calcul de prix, filtre, tri)
- ne charge pas de données dans un `useEffect`

> C'est la grille que j'utiliserai pour évaluer l'architecture du projet : **séparation stricte des responsabilités**.

06

L'état local et l'état global

Partager une donnée entre plusieurs écrans



Local ou global ?

	État local	État global
Exemple	Le texte d'une recherche, un menu ouvert	Les favoris, l'utilisateur connecté, le panier
Utilisé par	Un écran et ses enfants directs	Plusieurs écrans éloignés
Outil	`useState` dans un composant ou un hook	`Context` , ou une bibliothèque

> Faire descendre une prop à travers cinq composants qui ne s'en servent pas s'appelle le **prop drilling**. C'est le signe qu'un état global est nécessaire.

Créer un Context typé

src/context/favorites-context.tsx

```tsx
import { createContext, ReactNode, useState } from "react";

export type FavoritesContextValue = {
  favorites: string[];
  isFavorite: (id: string) => boolean;
  toggleFavorite: (id: string) => void;
};

export const FavoritesContext =
  createContext<FavoritesContextValue | null>(null);

export function FavoritesProvider({
  children,
}: {
  children: ReactNode;
}) {
  const [favorites, setFavorites] = useState<string[]>([]);

  const isFavorite = (id: string) => favorites.includes(id);
  const toggleFavorite = (id: string) =>
```

- Le type décrit ce que le Context **met à disposition**
- `null` par défaut : la valeur n'existe que sous un Provider
- Le **Provider** tient l'état et le transmet à tous ses descendants

Créer un Context typé (suite)

src/context/favorites-context.tsx

```
21      setFavorites((current) =>
22        current.includes(id)
23          ? current.filter((f) => f !== id)
24          : [...current, id],
25      );
26
27    return (
28      <FavoritesContext.Provider
29        value={{ favorites, isFavorite, toggleFavorite }}
30      >
31        {children}
32      </FavoritesContext.Provider>
33    );
34  }
```

Lire le Context avec un hook

src/hooks/use-favorites.ts

```ts
import { useContext } from "react";

import {
  FavoritesContext,
  FavoritesContextValue,
} from "@/context/favorites-context";

// Seule porte d'entrée vers les favoris partagés
export function useFavorites(): FavoritesContextValue {
  const context = useContext(FavoritesContext);
  if (!context) {
    throw new Error(
      "useFavorites doit être utilisé dans un FavoritesProvider",
    );
  }
  return context;
}
```

- Comme tout hook, il est rangé dans `src/hooks/`. Le fichier du Context ne contient que le Context et son Provider
- Le hook `useFavorites()` est la seule porte d'entrée. Les écrans n'importent jamais le Context directement
- L'erreur explicite signale un oubli du Provider dès le premier test
- Après le `if`, TypeScript sait que `context` n'est plus `null`

Fournir le Context à toute l'application

src/app/_layout.tsx

```tsx
import { Stack } from "expo-router";

import { FavoritesProvider } from "@/context/favorites-context";

export default function RootLayout() {
  return (
    <FavoritesProvider>
      <Stack screenOptions={{ headerTransparent: false }}>
        <Stack.Screen name="index" options={{ title: "Catalogue" }} />
      </Stack>
    </FavoritesProvider>
  );
}
```

- `_layout.tsx` enveloppe **tous les écrans** : c'est l'endroit du Provider
- Le `Stack` et ses options ne changent pas : le Provider l'entoure
- Chaque modification de la valeur affiche à nouveau **tous** les composants qui lisent ce Context. Ne mettez pas tout dedans

Un composant qui lit le Context

src/components/favorites-badge.tsx

```tsx
import { StyleSheet, Text } from "react-native";

import { useFavorites } from "@/hooks/use-favorites";

// Lit les favoris dans le Context : aucune prop à recevoir
export function FavoritesBadge() {
  const { favorites } = useFavorites();
  return (
    <Text style={styles.badge}>★ {favorites.length} favori(s)</Text>
  );
}

const styles = StyleSheet.create({
  badge: { fontSize: 16, fontWeight: "600", color: "#0B7A70" },
});
```

- Le badge importe le hook `useFavorites`, pas le Context
- **Aucune prop** : il lit les favoris directement, où qu'il soit placé sous le Provider
- Sans Context, il faudrait passer les favoris de parent en enfant jusqu'à lui

Le catalogue partage ses favoris

src/app/index.tsx

```tsx
import { FlatList, StyleSheet, Text, View } from "react-native";

import { FavoritesBadge } from "@/components/favorites-badge";
import { StateMessage } from "@/components/state-message";
import { StayCard } from "@/components/stay-card";
import { StayFiltersBar } from "@/components/stay-filters-bar";
import { useFavorites } from "@/hooks/use-favorites";
import { useColumns } from "@/hooks/use-columns";
import { useStayFilters } from "@/hooks/use-stay-filters";
import { useStays } from "@/hooks/use-stays";

// Écran : il assemble les contrôleurs (hooks) et les vues
// (composants)
export default function CatalogScreen() {
  const columns = useColumns();
  const { state, reload } = useStays();
  const stays = state.status === "success" ? state.stays : [];
  const { filters, updateFilters, visibleStays } =
    useStayFilters(stays);
  const { isFavorite, toggleFavorite } = useFavorites();
```

- Le catalogue importe le badge et le hook des favoris
- Les favoris ne sont plus un état local de l'écran : ils viennent du Context
- `<FavoritesBadge />` se met à jour à chaque favori ajouté, sans recevoir de props

Le catalogue partage ses favoris (suite)

src/app/index.tsx

```tsx

  if (state.status === "loading") return <StateMessage loading />;
  if (state.status === "error")
    return <StateMessage message={state.message} onRetry={reload} />;

  return (
    <View style={styles.container}>
      <View style={styles.header}>
        <Text style={styles.heading}>Séjours</Text>
        <FavoritesBadge />
      </View>
      <StayFiltersBar filters={filters} onChange={updateFilters} />
      <FlatList
        key={columns}
        numColumns={columns}
        data={visibleStays}
        keyExtractor={(stay) => stay.id}
        renderItem={({ item }) => (
          <StayCard
            stay={item}
```

Le catalogue partage ses favoris (suite)

src/app/index.tsx

```tsx
            isFavorite={isFavorite(item.id)}
            onToggleFavorite={toggleFavorite}
          />
        )}
        ListEmptyComponent={
          <Text style={styles.empty}>
            Aucun logement ne correspond.
          </Text>
        }
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    width: "100%",
    maxWidth: 1100,
    alignSelf: "center",
```

Le catalogue partage ses favoris (suite)

src/app/index.tsx

```
61      padding: 8,
62    },
63    header: {
64      flexDirection: "row",
65      justifyContent: "space-between",
66      alignItems: "center",
67      margin: 8,
68    },
69    heading: { fontSize: 24, fontWeight: "800" },
70    empty: { margin: 8, fontSize: 16 },
71  });
```

Et les bibliothèques d'état ?

Zustand

Un état global en quelques lignes, sans Provider. Seuls les composants concernés sont affichés à nouveau

Redux Toolkit

Le standard historique des grosses applications, plus verbeux, avec des outils de débogage poussés

TanStack Query

Spécialisé dans les **données du serveur** : cache, rechargement, état de chargement

Dans ce module, **Context suffit**. Une bibliothèque est un bonus du projet si vous savez expliquer pourquoi vous l'avez choisie.

Quiz

Un composant appelle `useFavorites()` mais l'application plante avec « useFavorites doit être utilisé dans un FavoritesProvider ». Pourquoi ?

> Le composant n'est pas à l'intérieur du `FavoritesProvider`. Il faut l'ajouter au-dessus, en général dans `_layout.tsx`.

Le texte d'un champ de recherche doit-il aller dans un Context ?

> Non. Il ne sert qu'à un écran : un état local suffit. Le Context est réservé à ce que plusieurs écrans partagent.

À retenir

- **Modèle** : des types et des fonctions pures dans `models/`, l'accès aux données dans `services/`
- Le **service** appelle l'API et convertit sa réponse en objets du modèle
- **Contrôleur** : des hooks personnalisés qui tiennent l'état et appellent les services
- **Vue** : des composants qui reçoivent des props et signalent des événements
- Un état en **union discriminée** rend les états absurdes impossibles
- Ce qui se calcule ne se stocke pas
- **Context** pour l'état partagé entre composants, exposé par un hook rangé dans `src/hooks/`

Des questions ?

Chris CHEVALIER
chevalier@chris-freelance.com

